CS 61A — Week 1

Solutions to the handout problems (code for the exercises is in `solutions/`)

P1. The operands of `sub` are evaluated left to right, each from the inside out.

1. `add(1, 2)` is `3`. Then `pow(2, 3)` is `8`.
2. `add(1, 1)` is `2`. Then `mul(2, 2)` is `4`.
3. `sub(8, 4)` is `4`.

Order in which the calls finish: `add(1, 2)`, `pow`, `add(1, 1)`, `mul`, `sub`. Value: `4`.

P2.

1. `a = 3`. `b = a * 2` binds `b` to `6`. `a = b - 1` binds `a` to `5`.
2. `a, b = b, a + b`: both right sides are evaluated first, with the old values: `6` and `5 + 6 = 11`. Then `a` is bound to `6` and `b` to `11`.

Displayed: `6` and then `11`.

P3.

1. `f` is bound to the built-in minimum function.
2. `min = max`: the name `min` is now bound to the maximum function.
3. `max = f`: the name `max` is now bound to the minimum function.
4. `min(1, 2)` is the maximum, `2`. `max(3, 4)` is the minimum, `3`.

Displayed: `5`.

P4.

1. `x = print(1, 2)`: displays `1 2`. `x` is bound to `None`.
2. `x`: the value is `None`, so the interpreter displays nothing.
3. `print(x)`: displays `None`.
4. `print(print(3), 4)`: the operand `print(3)` displays `3` and has value `None`; then the outer call displays `None 4`.
5. `print(7) + 1`: displays `7`; then `None + 1` is an `ERROR` (a `TypeError`).

P5.

1. `loud(loud(1))`: the inner call displays `got 1` and returns `2`. The outer call displays `got 2` and returns `4`. The interpreter displays `4`.
2. `quiet(quiet(1))`: the inner call displays `2` and returns `None` (no `return`). The outer call evaluates `2 * None`: `ERROR`.

Displayed: `got 1`, `got 2`, `4`, then `2`, then an error.

P6.

1. Global: `f` bound to `func f(x, y) [parent=Global]`; `x` bound to `3`.
2. `y = f(x, x + 1)`: operands evaluated in the global frame: `3` and `4`.
3. Frame `f1: f [parent=Global]`: `x` is `3`, `y` is `4`.
4. `x = x + y`: in `f1`, `x` becomes `7`. `y = x * 2`: in `f1`, `y` becomes `14`.
5. Return value `14 - 7 = 7`. Global `y` is bound to `7`. Global `x` is still `3`.

Displayed: `3 7`.

P7.

1. The `import` binds `add` to the library function; the `def` then rebinds `add` in the global frame to the new two-parameter function, which returns its second argument.
2. `x = add(1, 2)`: in `f1`, the parameter `add` is `1` and `mul` is `2`. Returns `2`. `print(x)` displays `2`.
3. `add(add, x)`: operands evaluated in the global frame: the function and `2`. In `f2`, `add` is the function and `mul` is `2`. Returns `2`.

Displayed: `2` and `2`.

P8. The `def` does not run the body, so no error there. `print(z)` displays `10`. The error (a `NameError`) happens when `h(z)` is called and the body evaluates `undefined_name`. So `10` is displayed first, then the error.